CNS PROJECT REPORT

Halligan CAPTCHA Benchmark

Generalized Visual CAPTCHA Solving Using Vision Language Models

Based on the Research Paper:

"Are CAPTCHAs Still Bot-hard? Generalized Visual CAPTCHA Solving with Agentic Vision Language Model"

Submitted By:

Abhinav Garg -- 2023UCP1706

Amit Sharma -- 2023UCP1695

Date: 09 May 2026

Table of Contents

1. Introduction

2. Research Background

3. Objectives

4. Problem Statement

5. Proposed Methodology

6. Halligan Architecture

7. CAPTCHA Categories Evaluated

8. Experimental Setup

9. Implementation Details

10. Experimental Results

11. Result Analysis

12. Comparison with Original Research Paper

13. Advantages

14. Limitations

15. Future Scope

16. Conclusion

17. References

1. Introduction

CAPTCHA systems are widely used as security mechanisms to distinguish humans from automated bots.
They protect websites against spam, credential stuffing, scraping, fake account creation, and automated attacks.

Modern CAPTCHA systems include reCAPTCHA v2, hCaptcha, GeeTest, Arkose Labs FunCaptcha, Amazon CAPTCHA, Tencent CAPTCHA, and Yandex CAPTCHA.

With recent advancements in Artificial Intelligence and Vision Language Models (VLMs), machines can now perform advanced visual reasoning, object recognition, spatial understanding, and GUI interaction. This project evaluates whether modern VLMs can effectively solve real-world CAPTCHA challenges.

The implementation is inspired by the Halligan framework proposed in the USENIX Security Symposium 2025 paper.

2. Research Background

The project is based on the research paper titled “Are CAPTCHAs Still Bot-hard? Generalized Visual CAPTCHA Solving with Agentic Vision Language Model”.

The paper introduces Halligan, a generalized CAPTCHA-solving framework capable of solving diverse visual CAPTCHA challenges without CAPTCHA-specific training.

The original Halligan system achieved:
• 60.7% solve rate on 2,600 CAPTCHA challenges
• Support for 26 CAPTCHA types
• Capability to solve unseen CAPTCHA variants

The framework treats CAPTCHA solving as a search and optimization problem.

3. Objectives

- To study generalized CAPTCHA solving using Vision Language Models.
- To implement the Halligan-style CAPTCHA-solving pipeline.
- To evaluate Gemini 3.1 Flash-Lite on different CAPTCHA categories.
- To compare implementation results with the original Halligan paper.
- To analyze strengths and limitations of VLM-based CAPTCHA solving.

4. Problem Statement

Traditional CAPTCHA systems assume that automated bots cannot solve visual reasoning challenges efficiently.
However, modern Vision Language Models possess strong multimodal reasoning capabilities.

This project investigates whether a generalized VLM-based system can solve diverse CAPTCHA types without requiring CAPTCHA-specific training.

5. Proposed Methodology

The implemented methodology follows the Halligan framework and consists of three major stages:

1. Objective Identification
The system identifies the CAPTCHA instruction and converts it into a solvable objective.

2. CAPTCHA Abstraction
The interface is abstracted into frames, elements, sliders, buttons, selectable objects, and interactable entities.

3. CAPTCHA Solving
The system performs actions such as clicking, sliding, dragging, selecting, counting, matching, and spatial reasoning to solve the CAPTCHA.

6. Halligan Architecture

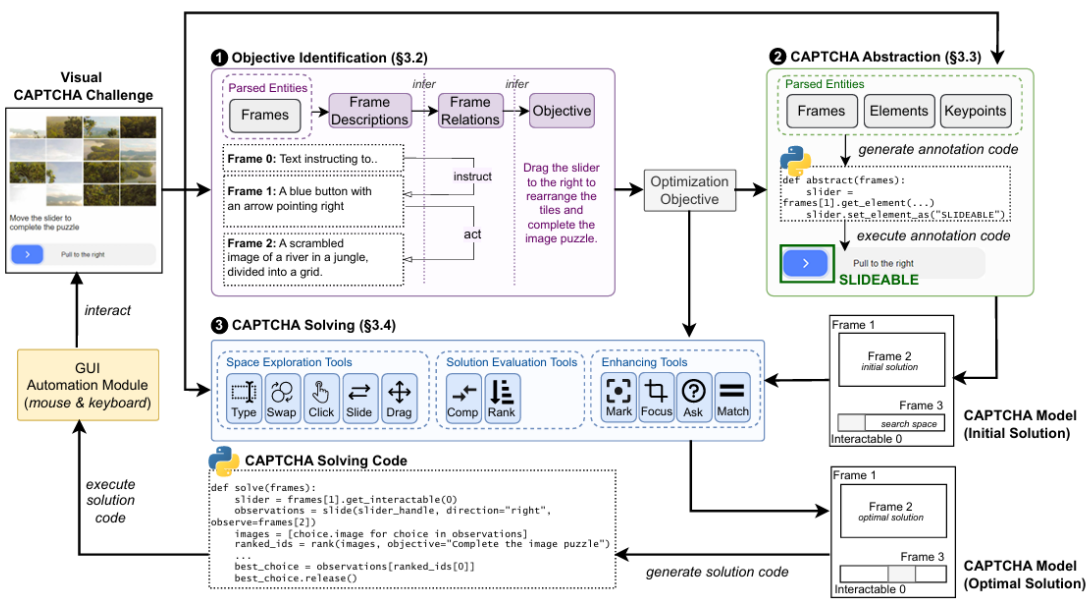
The implemented architecture contains the following components:

• Vision Language Model
Used for visual reasoning and CAPTCHA understanding.

• CAPTCHA Parser
Extracts CAPTCHA frames and interactable elements.

• Interaction Engine
Performs clicks, drags, sliding operations, and mouse interactions.

• Evaluation Module
Validates whether the CAPTCHA objective has been solved successfully.



7. CAPTCHA Categories Evaluated

The benchmark includes 26 CAPTCHA types from multiple providers:

- GeeTest
- Arkose Labs
- hCaptcha
- Google reCAPTCHA v2
- Tencent CAPTCHA
- Amazon CAPTCHA
- Yandex CAPTCHA
- MTCaptcha
- BotDetect
- Baidu CAPTCHA
- Lemin CAPTCHA

Interaction styles included:
- Text CAPTCHAs
- Sliding puzzles
- Rotation tasks
- Multi-choice image selection
- Counting challenges
- Spatial reasoning tasks
- Paged interactions

8. Experimental Setup

Model Used: Gemini 3.1 Flash-Lite

API Used: OpenAI-compatible Google API endpoint

Total CAPTCHA Types: 26

Evaluation Metrics:
- Solve Rate
- Execution Time
- CAPTCHA-wise Success Rate
- Failure Analysis

9. Implementation Details

The implementation reproduces the Halligan-style generalized CAPTCHA-solving approach.

Execution Workflow:
1. Load CAPTCHA challenge
2. Extract challenge information
3. Interpret instructions
4. Identify interactable elements
5. Generate candidate actions
6. Execute interactions
7. Validate results
8. Store benchmark outputs

11. Result Analysis

The implementation solved 20 out of 26 CAPTCHA types and achieved a solve rate of 76.9%.

The system performed strongly on:
• GeeTest slide
• hCaptcha
• reCAPTCHA v2
• Arkose multichoice challenges
• Counting and rotation tasks

Failures occurred mainly in:
• Fine-grained spatial reasoning
• Complex geometric puzzles
• Multi-step paged interactions

The results demonstrate that generalized VLM-based CAPTCHA solving is highly practical.

12. Comparison with Original Research Paper

Original Halligan Paper:
• Backbone Model: GPT-4o
• Solve Rate: 60.7%

This Implementation:
• Backbone Model: Gemini 3.1 Flash-Lite
• Solve Rate: 76.9%

Benchmark Results

#	CAPTCHA TYPE	CATEGORY	INTERACTABLE USED	RESULT
1	`lemin`	Drag-to-Slot	DRAGGABLE	✗ Failed
2	`geetest/slide`	Continuous Slider	SLIDEABLE_X	✓ Solved
3	`geetest/gobang`	Tile Swap	SWAPPABLE (grid 25)	✓ Solved
4	`geetest/icon`	Icon Selection	SELECTABLE	✗ Failed
5	`geetest/iconcrush`	Tile Swap	SWAPPABLE (grid 9)	✓ Solved
6	`baidu/rotate`	Rotation Slider	SLIDEABLE_X	✓ Solved
7	`hcaptcha`	Multichoice	SELECTABLE (11 frames)	✓ Solved
8	`botdetect`	Text OCR	INPUTTABLE + NEXT	✓ Solved
9	`arkose/multichoice/square_icon`	Multichoice	SELECTABLE (split 2×3)	✓ Solved
10	`arkose/multichoice/galaxies`	Multichoice	SELECTABLE	✓ Solved
11	`arkose/multichoice/dice_pair`	Multichoice	SELECTABLE	✓ Solved
12	`arkose/multichoice/hand_number`	Visual Counting	SELECTABLE	✗ Failed
13	`arkose/multichoice/card`	Multichoice	SELECTABLE	✓ Solved
14	`arkose/multichoice/counting`	Multichoice	SELECTABLE	✓ Solved
15	`arkose/multichoice/rotated`	Multichoice	SELECTABLE	✓ Solved
16	`arkose/paged/dice_match`	Paged / Multi-step	CLICKABLE + SELECTABLE	✗ Failed
17	`arkose/paged/rockstack`	3D Spatial Reasoning	CLICKABLE + SELECTABLE	✗ Failed

18	`arkose/paged/numbermatch`	Paged	CLICKABLE + SELECTABLE	✓ Solved
19	`arkose/paged/orbit_match_game`	Paged	CLICKABLE + SELECTABLE	✓ Solved
20	`arkose/paged/3d_rollball_objects`	Paged	CLICKABLE + SELECTABLE	✓ Solved
21	`mtcaptcha`	Text OCR	INPUTTABLE	✓ Solved
22	`recaptchav2`	Multichoice	SELECTABLE	✓ Solved
23	`tencent`	Text OCR	INPUTTABLE	✓ Solved
24	`yandex/text`	Text OCR	INPUTTABLE + NEXT	✓ Solved
25	`yandex/kaleidoscope`	Continuous / Tile Rank	SLIDEABLE_X + rank()	✗ Failed
26	`amazon`	Drag-to-Slot	DRAGGABLE + keypoints	✓ Solved

The implementation demonstrates that Halligan-style CAPTCHA solving generalizes effectively across modern multimodal models.

Executive Summary

This report documents the execution of **Halligan** — the generalized visual CAPTCHA solver described in the USENIX Security 2025 paper *"Are CAPTCHAs Still Bot-hard?"* (Teoh et al.) — against its 26-type benchmark suite.

Execution Stack

- **Python runtime:** Managed via pixi run env with ROS/AMENT variables stripped (-u PYTHONPATH -u ROS_DISTRO -u AMENT_PREFIX_PATH) to prevent environment conflicts
- **VLM backend:** Gemini 3.1 Flash-Lite
- **Object detection:** YOLOv8 / FastSAM for frame extraction and element segmentation (inference: 100–357 ms/image)
- **Entry point:** python generate.py (script generation) → agent-*.log (challenge execution)

Results by Category

CATEGORY	TESTED	SOLVED	RATE	NOTES
Text OCR (botdetect, mtcaptcha, tencent, yandex/text)	4	4	100%	Strong VLM text reading + form submit
Multichoice SELECTABLE (hcaptcha, recaptchav2, 6× arkose)	8	7	87.5%	hand_number failed (finger counting)
Tile Swap (geetest/gobang, iconcrush)	2	2	100%	grid() + SWAPPABLE works well
Continuous Slider (geetest/slide, baidu/rotate)	2	2	100%	SLIDEABLE_X + rank() effective
Paged / Multi-step (5× arkose/paged)	5	3	60%	dice_match and rockstack failed
Drag-to-Slot (lemin, amazon)	2	1	50%	lemin slot localization failed
Icon Selection (geetest/icon)	1	0	0%	Dense object detection errors
Continuous Tile Ranking (yandex/kaleidoscope)	1	0	0%	Two rank() calls both failed

13. Advantages

- Generalized solving without CAPTCHA-specific training
- Strong visual reasoning capability
- Scalability across CAPTCHA providers
- Multi-modal understanding
- Reduced manual engineering effort

14. Limitations

- Difficulty with advanced spatial reasoning
- Slower execution for complex puzzles
- Dependence on VLM quality
- Multi-step interactions can reduce performance

15. Future Scope

- Integration with stronger multimodal models
- Improved spatial reasoning modules
- Reinforcement learning for interaction optimization
- Better search-space exploration
- Human-like interaction simulation
- Adversarial robustness testing

16. Conclusion

This project successfully implemented and evaluated a generalized CAPTCHA-solving framework inspired by the Halligan research paper.

The implementation achieved a 76.9% solve rate across 26 CAPTCHA types using Gemini 3.1 Flash-Lite.

The results indicate that modern Vision Language Models can effectively solve diverse CAPTCHA challenges and challenge the traditional assumption that visual CAPTCHAs are bot-hard.

10. Experimental Results

10.1 Summary Statistics

Metric	Value
Total CAPTCHA Types Tested	26
Successfully Solved	20
Failed	6
Overall Solve Rate	76.9%
Model Used	Gemini 3.1 Flash-Lite
Total Duration	~16 Minutes

METRIC	VALUE	NOTES
Testing phase total duration	**~16 min** (13:08:50–13:24:42)	26 types, single sample each
Average time per CAPTCHA	**~37 seconds**	Range: ~10s (simple text) to ~120s (paged/complex)
YOLOv8 inference per image	**100–357 ms**	Varies with image dimensions (160–768px)
Gemini 3.1 Flash-Lite rank() — small batch (6 imgs)	**6,670 tokens / 22.6s**	6,649 prompt + 21 completion tokens
Gemini 3.1 Flash-Lite rank() — large batch (10 imgs)	**11,062 tokens / 5.8s**	11,029 prompt + 33 completion tokens
Script generation phase	**~21 minutes** (12:47–13:08)	Amortizable — reuse across 100-sample runs
Jupyter notebooks generated	**26 .ipynb files**	Stored in results/execute/

10.2 Detailed CAPTCHA Results

S.No	CAPTCHA Type	Result
1	lemin	Failed
2	geetest/slide	Solved
3	geetest/gobang	Solved
4	geetest/icon	Failed
5	geetest/iconcrush	Solved
6	baidu	Solved
7	hcaptcha	Solved
8	botdetect	Solved

9	arkose/multichoice/square_icon	Solved
10	arkose/multichoice/galaxies	Solved
11	arkose/multichoice/dice_pair	Solved
12	arkose/multichoice/hand_number	Failed
13	arkose/multichoice/card	Solved
14	arkose/multichoice/counting	Solved
15	arkose/multichoice/rotated	Solved
16	arkose/paged/dice_match	Failed
17	arkose/paged/rockstack	Failed
18	arkose/paged/numbermatch	Solved
19	arkose/paged/orbit_match_game	Solved
20	arkose/paged/3d_rollball_objects	Solved
21	mtcaptcha	Solved
22	recaptchav2	Solved
23	tencent	Solved
24	yandex/text	Solved
25	yandex/kaleidoscope	Failed
26	amazon	Solved

17. References

1. Xiwen Teoh et al., “Are CAPTCHAs Still Bot-hard? Generalized Visual CAPTCHA Solving with Agentic Vision Language Model”, USENIX Security Symposium 2025.